28. b. Singly Linked List – Destroy List – Space Complexity and Algorithm.

***Program*: –**

```
void destroyList(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }

    Node **current = head;
    Node *next;
    while (*current != nullptr)
    {
        next = (*current)->next;
        free(*current);
        *current = next;
    }
}
…..
case 27:
        {
            cout << endl;
            cout << endl;
            cout << "Destroying List....." << endl;
            destroyList(&head);
            cout << "List Destroyed." << endl;
            cout << endl;
            cout << endl;
```

Space Complexity Analysis

Auxiliary Space

Auxiliary space is the extra or temporary memory used by a function, not including the input data itself.

In this version of the destroyList function, the new variables created are:

*Node **current: A pointer to a pointer, used to iterate through the list by advancing the head pointer itself. The memory for a pointer (even a double pointer) is constant.*

*Node *next: A single pointer to store the address of the next node before the current one is deleted. This also requires a constant amount of space.*

The function's main purpose is to free memory. Since the amount of extra memory used is fixed (just two pointers), the auxiliary space complexity is O(1).

Auxiliary Space: O(1)

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter-only Definition

*This view considers only the space taken by the function's direct parameters. The function signature is void destroyList(Node **head).*

The function takes a single parameter, head, which is a pointer to a pointer. A pointer occupies a fixed, constant amount of memory.

Under this definition, the input space is constant.

Input Space (Parameter-only): O(1)

2. Full-Data Definition

*This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list pointed to by *head.*

If the linked list contains n nodes, the total memory it occupies is proportional to n.

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): O(n)

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\boldsymbol{Space} = \underbrace{\boldsymbol{O}(\mathbf{1})}_{\boxed{\textit{Auxiliary}}} + \underbrace{\boldsymbol{O}(\mathbf{1})}_{\boxed{\textit{Input}}} = \boldsymbol{O}(\mathbf{1}).$$

- **Holistic Interpretation (Full-data input):**

$$\boldsymbol{Space} = \underbrace{\boldsymbol{O}(\mathbf{1})}_{\boxed{\textit{Auxiliary}}} + \underbrace{\boldsymbol{O}(\boldsymbol{n})}_{\boxed{\textit{Input}}} = \boldsymbol{O}(\boldsymbol{n}).$$

In most contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is O(1). The while loop affects the function's time complexity (O(n)), but not its space complexity.

Algorithm: Destroy List (by Reference)

DESTROY_LIST (START)

1. $[CHECK\ IF\ LIST\ IS\ EMPTY]$

   $If\ \ START\ =\ NULL, then:$

   $Write: "List\ is\ empty."$

   $EXIT.$

   $[End\ of\ If\ structure.]$

2. $[INITIALIZE\ POINTER]$

   $Set\ CURRPTR := \ START. [Pointer\ to\ the\ current\ node].$

   $Declare\ NEXT_NODE. [Pointer\ to\ save\ the\ next\ node's\ address].$

3. $[TRAVERSE\ THE\ LIST\ AND\ FREE\ EACH\ NODE]$

   $Repeat\ while\ CURRPTR\ \neq\ NULL:$

   $a. Set\ NEXT_NODE := \ LINK[CURRPTR] \begin{pmatrix} Save\ the\ address\ of\ the\ next\ node \\ before\ deleting\ the\ current\ one \end{pmatrix}.$

   $b. free(CURRPTR) \begin{pmatrix} Deallocate\ the\ memory \\ of\ the\ current\ \ node \end{pmatrix}.$

   $c. Set\ CURRPTR := \ NEXT_NODE \begin{pmatrix} Update\ the\ original\ START\ pointer\ to\ point\ to\ the \\ next\ node, \\ effectively\ shortening\ the\ list\ from\ the\ front \end{pmatrix}.$

   $[END\ of\ Loop]$

4. $Exit$